

Operators and Casting

INTRODUCTION TO JAVA



Jim White
Java Developer

Operators

- Arithmetic (`-` , `+` , `-` , `*` , `/` , `%` , `++` , `--`)
- Comparison or Relational (`==` , `!=` , `>` , `<` , `>=` , `<=`)
- Logical (`&&` , `||` , `!`)
- Assignment (`=` , `+=` , `-=` , `*=` , `/=` , `%=` , `<<=` , `>>=` , `&=` , `^=` , `|=`)
- Bitwise (`&` , `|` , `^` , `~` , `<<` , `>>` , `>>>`)
- Miscellaneous (`?:` , `instanceof`)

Arithmetic operators

- Addition `+`
- Multiplication `*`
- Subtraction `-`
- Division `/`
- Increment `++` : add one to the value
- Decrement operators `--` : remove one from the value

```
int a = 5 + 6;    //a is 11
int b = a * 7;    //b is 77
int c = b - a;    //c is 66
double d = c / 6; //d is 11.0
```

```
int y = 5;
y++;   // Now y = 6
y--;   // Now y = 5 again
```

Comparison

- Comparing things defines a *relation* or *comparison*
 - Comparing whether one is bigger, smaller, or the same
 - Comparisons take the form of a question
 - Is **A** bigger than **B**?
 - The answer is **true** or **false** (a boolean)

Comparison operators

- Comparison questions use operators
 - Shortcuts to represent bigger, smaller, equal to relationships
- Makes coding comparison questions faster and more concise

Relation	Operator	Example Use	Example Result
Greater	>	6 > 5	true
Less	<	7 < 6	false
Equal to	==	5 == 5	true
Greater or equal to	>=	6 >= 5	true
Less or equal to	<=	7 <= 7	true
Not equal to	!=	6 != 6	false

Java comparison operators

- Comparison operators: `==` , `!=` , `>` , `<` , `>=` , `<=`
 - Results in a `true` or `false` boolean when used

```
boolean x = 5 <= 6; // x is true since 5 is less than or equal to 6
boolean y = 5 != 6; // y is true since 5 does not equal 6
int a = 8;
int b = 2;
boolean z = a > b; // z is true since a value is greater than b value
```

Comparison among char

- Use `>` , `<` , `>=` , `<=` , and `!=` to compare integer and floating point numbers
- Comparison operators can also be used with characters

```
char medium = 'M';  
char large = 'L';  
System.out.println(medium < large); // Outputs false  
System.out.println(medium >= large); // Outputs true
```


Equals operator

- `=` is assignment, but `==` is the equal-to operator
 - Equal operator `==` returns a boolean result

```
int x = 5; // Assignment
int y = 6; // Assignment
boolean isEqual = x == y; //Equal operator for comparison
System.out.println(x == y); // Prints out false
```

- Common mistake in Java: using `=` in place of `==`

```
int a = 5; // Assigns 5 to a
boolean b = a == 5; // Assigns true to b
boolean c = a = 5; // Results in an error!
```


Assignment operators

- Equal operator
 - Assigns the value on the right to the variable on the left

```
int x = 7; // Assigns 7 to x
```

- Additional assignment operators
 - `+=` , `-=` , `*=` and `/=`
 - Shortcuts to assignment with other operations

```
int a = 5;  
int b = 2;  
a += b; // Same as a = a + b, a is now 7  
System.out.println(a -= 3); // Prints 4
```

Operators covered

- **Covered**

- Arithmetic (`-` , `+` , `-` , `*` , `/` , `%` , `++` , `--`)
- Comparison or Relational (`==` , `!=` , `>` , `<` , `>=` , `<=`)
- Assignment (`=` , `+=` , `-=` , `*=` , `/=` , `%=` , `<<=` , `>>=` , `&=` , `^=` , `|=`)

- **Covered Later**

- Logical (`&&` , `||` , `!`)

- **Not Covered/Rarely used**

- Bitwise (`&` , `|` , `^` , `~` , `<<` , `>>` , `>>>`)
- Miscellaneous (`?:` , `instanceof`)

Casting

- *Type casting* or just *casting* converts one data type to another
- Casting from a smaller type to a larger is done automatically by Java
 - Example: turning a short into a long
 - No special syntax is required

```
short balance = 100;  
long longBalance = balance; //longBalance is 100 but now as a long
```

Narrowing casting

- *Narrowing* casting converts a larger type into a smaller type
 - Example: turning a double into a float
- Done to conserve space and to match the data's type to the variable type
- Narrowing requires special syntax
 - Use `( )` with a type to perform narrowing.

```
double y = 5.5d; // double uses 64 bits of memory  
float x = (float) 5.5d; // float only uses 32 bits of memory
```

Let's practice!

INTRODUCTION TO JAVA

Strings

INTRODUCTION TO JAVA



Jim White
Java Developer

What are Strings?

- Strings are a collection of characters between double quotes.
 - Characters in a String include letters, numbers, punctuation, spaces, etc.
- Some examples of Strings

```
"Today is a good day"
```

```
"12456"
```

```
":)"
```

```
"abc789 # $"
```

- Even a single character in double quotes `"` is a String (ex: `"H"`)
 - Versus char primitives surrounded by single quotes `'` (ex: `'H'`)

More about Strings

- **String** is a type in Java
 - A *non-primitive* type
 - Primitive types are data types from which all other data types are constructed
 - Strings are constructed from a sequence of primitive char
- Strings have operators and methods
 - Allow work to be done on Strings
- Strings are *arrays* of characters

Creating Strings

- Use String variables to hold Strings
- The syntax to define a String variable and assign it a String *literal*
 - **String** *variable* = "text here";

```
String helloWorld = "Hello World";
```

- "Hello World" is a String literal

String output

- Output Strings just like primitives using `System.out.println`

```
System.out.println("Its a whole new world"); // Output a String literal
String helloWorld = "Hello World";
System.out.println(helloWorld); // Output a String variable's value
```

```
Its a whole new world
Hello World
```

String concatenation

- Strings have a special operator: `+`
 - This is **not** the addition operator used in primitive type arithmetic (ex: `4 + 5`)
- Use `+` operator with Strings to concatenate them.
 - Or concatenate anything (like a primitive) to a String

```
String hello = "Hello";  
String world = " World";  
String helloWorld = hello + world;  
String manyHelloWorlds = helloWorld + '*' + 10;  
System.out.println(helloWorld);  
System.out.println(manyHelloWorlds);
```

```
Hello World  
Hello World*10
```

String methods

- `main` is the only method we know
 - Methods are a way to organize code in classes
 - Methods also perform operations on data
- String methods perform operations on Strings
 - String has more than 50 methods

.length() method

- `.length( )` returns the number of characters in a String

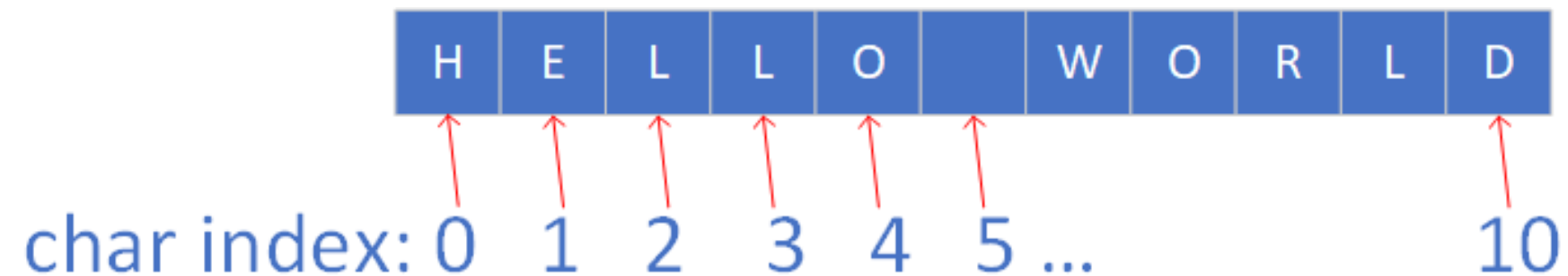
```
String helloWorld = "Hello World";  
int size = helloWorld.length();  
System.out.println(size); // Outputs 11
```

charAt() method

- `charAt(int index)` gives the character at a given index on a String

```
String helloWorld = "Hello World";  
char whatAtTwo = helloWorld.charAt(2);  
System.out.println(whatAtTwo); //Outputs the char l
```

- Java starts counting at 0, not 1
 - Java is a "zero-based" index language
 - `charAt(2)` asks for the 3rd character



¹ https://en.wikipedia.org/wiki/Zero-based_numbering

equals() method

- `==` operator between Strings is allowed
 - `==` is `true` if the Strings occupy the same memory location
- Use `equals(String input)` to determine if two Strings have the same characters

```
String x = "Hello World";  
String y = "Hello World";  
System.out.println(x==y); //true if in the same memory location  
System.out.println(x.equals(y)); //true because the Strings match characters
```


toUpperCase()/toLowerCase() methods

- `toUpperCase()` returns a new String with all the characters in uppercase
- `toLowerCase()` returns a new String with all the characters in lowercase

```
String x = "HelloWorld"  
System.out.println(x.toUpperCase()); // Outputs HELLOWORLD  
System.out.println(x.toLowerCase()); // Outputs helloworld
```


String Cheat Sheet

- Create Strings: `String variable = "text here";`
- Concatenate Strings: `"string1" + "string2"`
- String Methods

Description	Method Example
String length	<code>"a string".length()</code>
Char in a String	<code>"a string".charAt(<i>index</i>)</code>
2 Strings equal	<code>"a string".equals("a string")</code>
String in lowercase	<code>"a string".toLowerCase()</code>
String in uppercase	<code>"a string".toUpperCase()</code>

Let's practice!

INTRODUCTION TO JAVA

Arrays

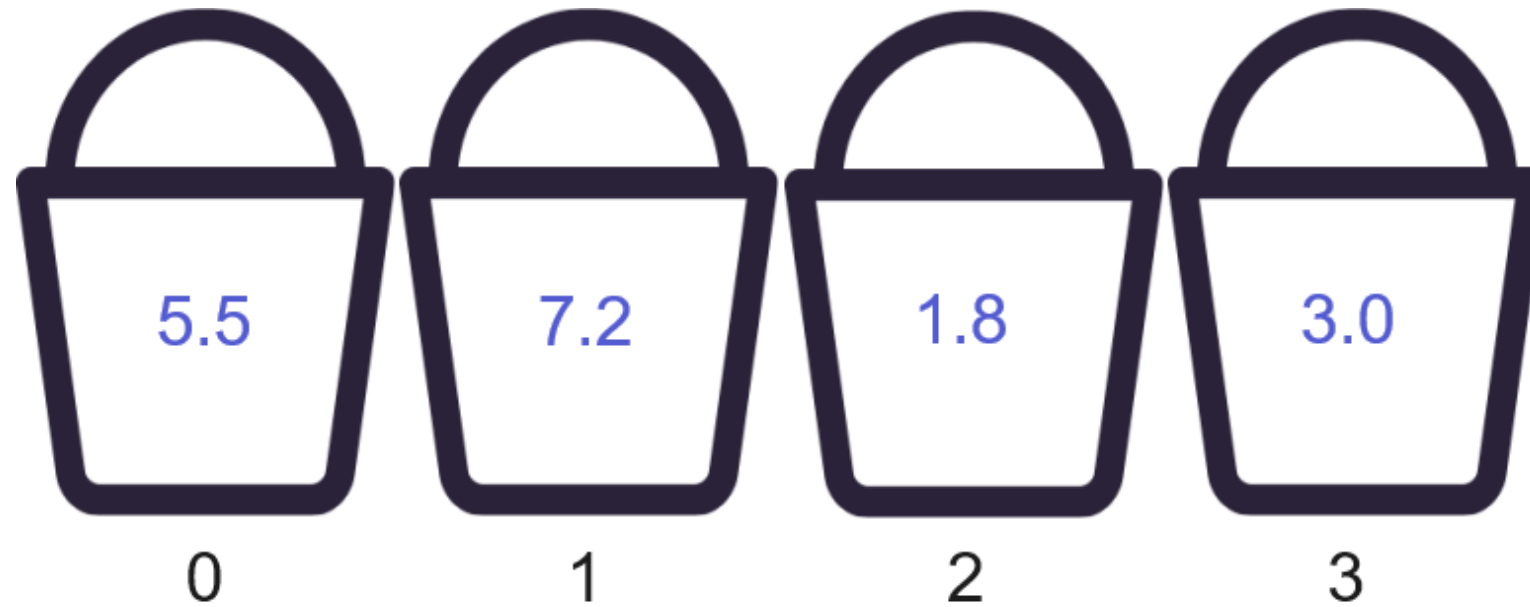
INTRODUCTION TO JAVA



Jim White
Java Developer

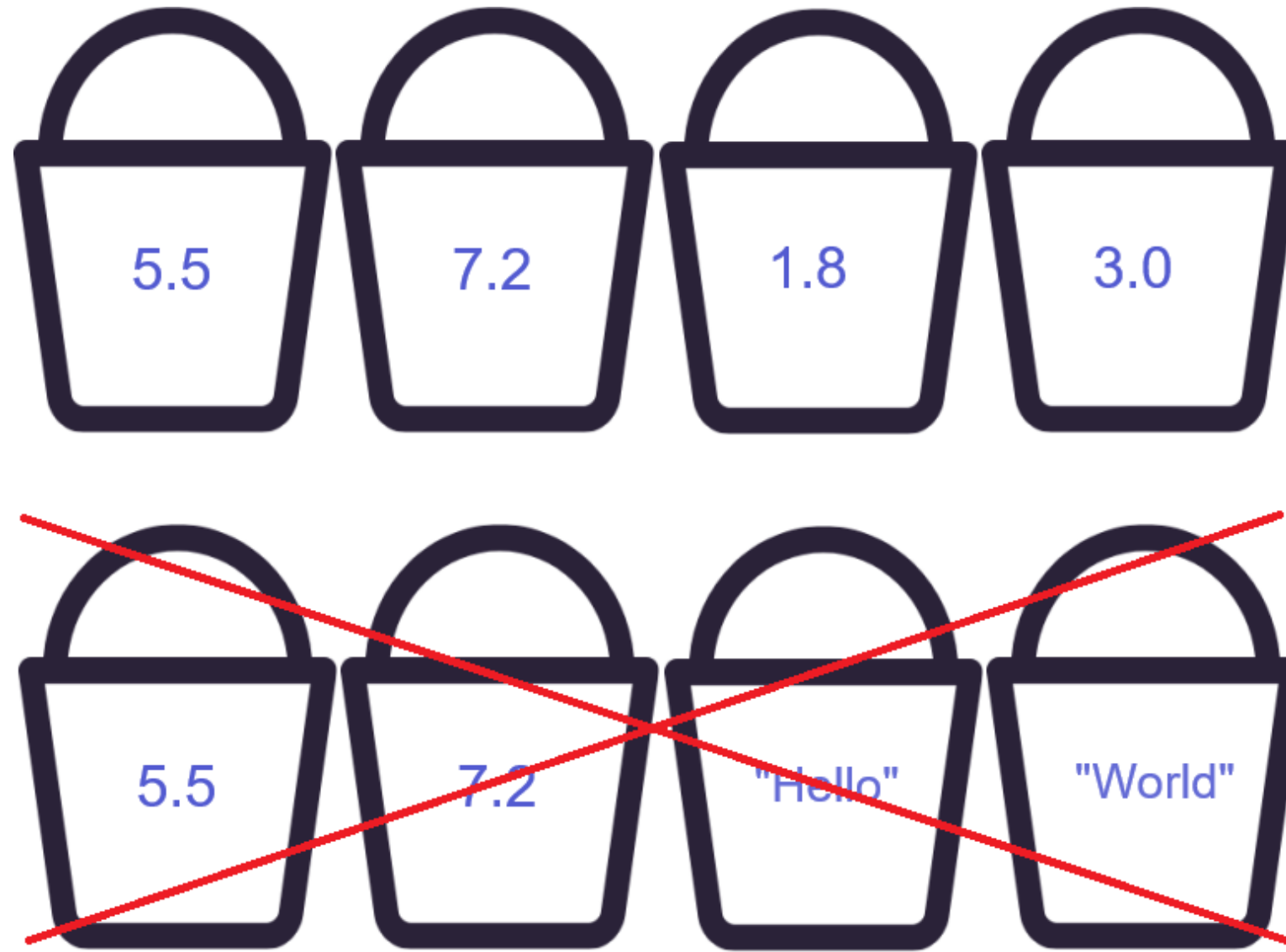
What's an array

- An array is a sequence of data values
 - Like a set of buckets, each bucket holding a value



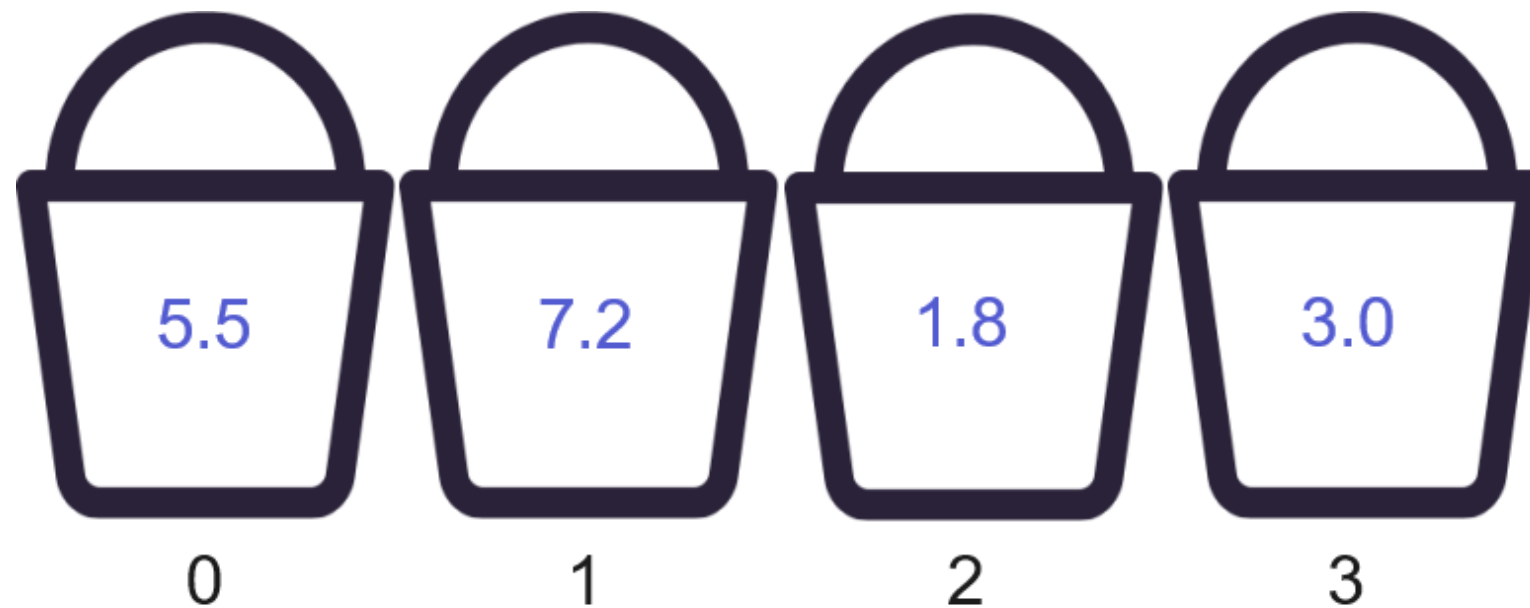
Array values

- The buckets could hold any type values (int, double, Strings, etc.)
 - But the values must be the same type



Array order

- Values in the array are ordered or *indexed*
 - The index is a means to get values in/out of the buckets
 - "Put 7.2 in the 2nd bucket, and get the value 1.8 out of the 3rd bucket"
- The index starts at 0 (not 1)
 - Java is a zero-index based language.



More about arrays

- Arrays are another data type
 - Store multiple values (primitives, Strings, etc.) under a single variable
- Arrays are *fixed* size
 - Elements in the array can change, but its size cannot
 - Once set, the array cannot change

Array variable syntax

- Arrays are defined with a special syntax: `[ ]`
 - Called *square bracket notation*
- Array variables must be typed
 - Sets the type of values that go into the array buckets
 - **All** values in the array must be of that type
- Declaring an array variable can be done in one of two ways

```
int[] myScores;    // Preferred
int myScores[];    // Non-preferred
```


Arrays of any type

- Arrays can be of any type

```
double[] temperatures;  
char[] grades;  
String[] names;
```

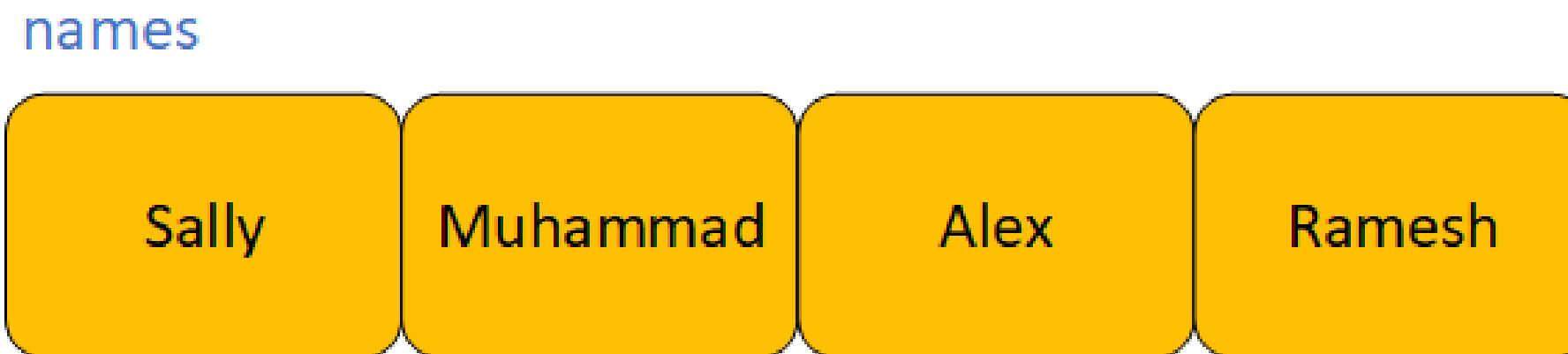
- Array values must be *homogeneous* (the same)
 - Therefore the array type applies to all values in the array

Filling the array

- Use special notation to set the values of the array
 - A list of values surrounded by `{ }`

```
int[] myScores = {75, 91, 88};  
String[] names = {"Sally", "Muhammad", "Alex", "Ramesh"};
```

- Once the array contents are set, the array size is also set

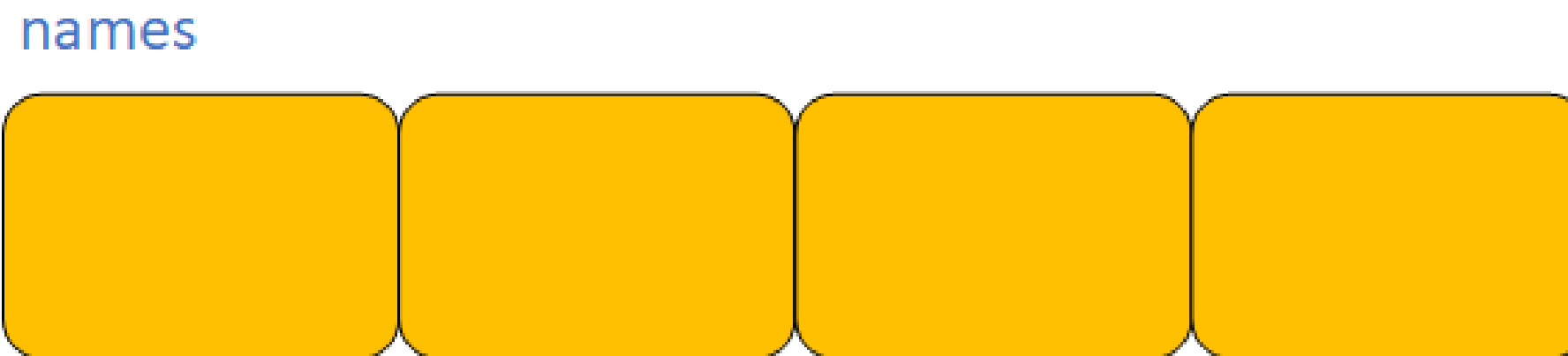


Declaring array size

- Alternately, when declaring the array variable, the size can be set
 - Requires use of *new* operator
 - *type* variable-name = **new** *type*[size];

```
int[] myScores = new int[3];  
String[] names = new String[4];
```

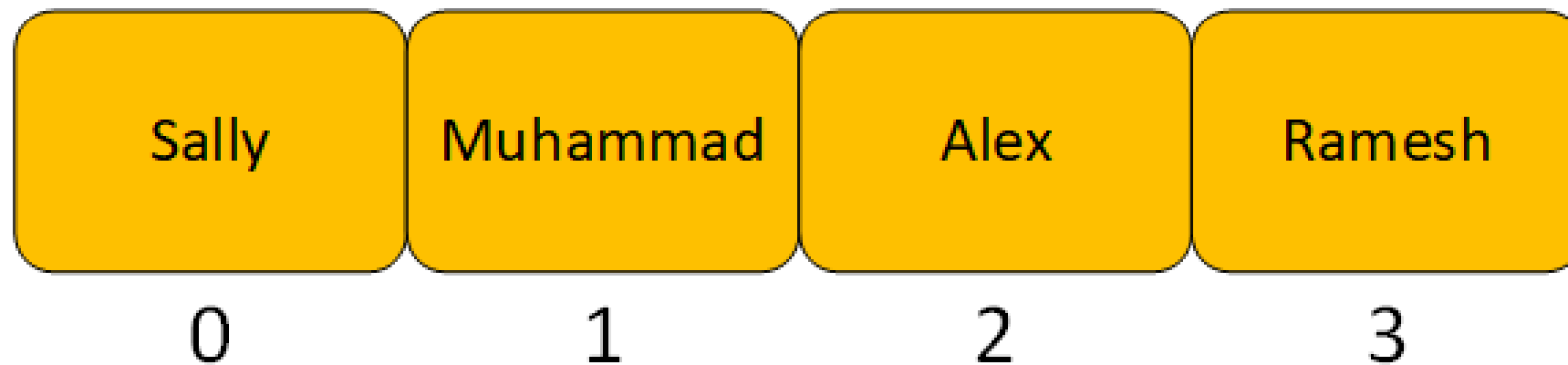
- Here, values are not provided, but the array size is set
 - Values can later be provided



Accessing array elements

- Arrays are indexed from 0 to the length of the array - 1
 - Values in this `names` String array are indexed from 0 to 3

`names`



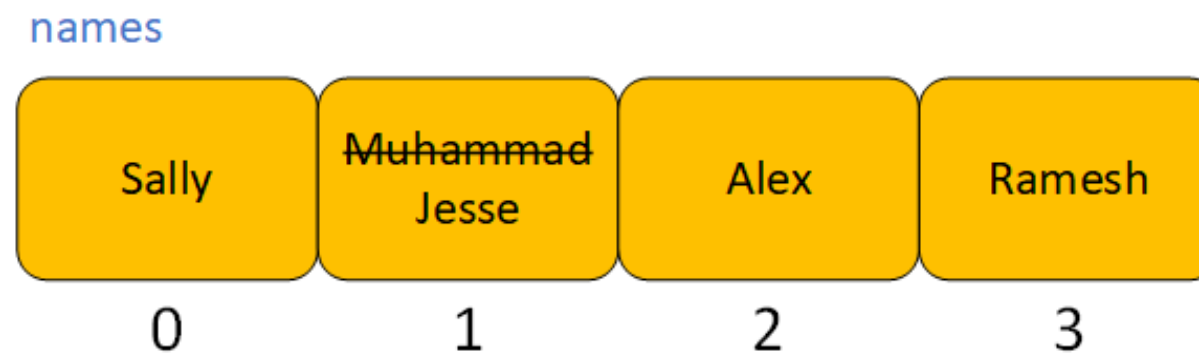
- Access any value in an array using square bracket notation
 - Use the variable name and the value's index in square brackets

```
String myName = names[1]; // myName is set to Muhammad
```

Setting array elements

- Also use square bracket notation to set or change any value in an array

```
names[1] = "Jesse";
```



- Be careful, trying to get or set the value outside of the array indexes will cause an exception

```
name[4] = "Jim"; // Causes an exception
```

```
java.lang.ArrayIndexOutOfBoundsException: Index 4 out of bounds for length 4
```

Filling an empty array

- Here's how to set the values of an array created with a size but with no values

```
String[] names = new String[4];  
names[0] = "Sally";  
names[1] = "Muhammad";  
names[2] = "Alex";  
names[3] = "Ramesh";
```

Array length

- Arrays have a *property* to get their length
 - Properties are information about the data type
- Use dot notation with `length` to get the array's length property

```
String[] names = {"Sally", "Muhammad", "Alex", "Ramesh"};  
int namesLength = names.length; // namesLength would be 4
```

- The length is useful for working with and performing operations on the array

Length Property vs Method

- Often a point of confusion for new Java developers
 - String has a length method (`"HelloWorld".length()`)
 - Arrays have a length property (`myScores.length`)
 - Both do the same kind of work
- All programming languages have some idiosyncrasies
 - Design decisions that do not always seem to make sense
 - This is one of Java's

Strings as arrays

- Strings are special arrays
 - Arrays of char that do not use `[]` notation
- Use `charAt()` method to get a char in the String at an index
- Use `toCharArray()` method to convert it to an array of char

```
String myName = "Jacob";  
char thirdLetter = myName.charAt(2); // 'c'  
char[] myNameChars = myName.toCharArray();  
// above same as char[] myNameChars = {'J', 'a', 'c', 'o', 'b'};  
char fourthLetter = myNameChars[3]; // 'o'
```

Array Cheat Sheet

- Defining an array variable

```
int[] myScores;    // Preferred
int myScores[];    // Non-preferred
```

- Defining and filling an array

```
int[] myScores = {75, 91};
```

- Defining and later filling an array

```
int[] myScores = new int[2];
myScores[0] = 75;
myScores[1] = 91;
```

- Getting the array length

```
myScores.length
```

Let's practice!

INTRODUCTION TO JAVA

Creating and Calling Methods

INTRODUCTION TO JAVA



Jim White
Java Developer

What are Java methods?

- Form the basis of all Java applications
- Methods are also called *functions* or *operations*
- Named building blocks of Java code
- Define reusable blocks of instructions

Language provided methods

- Java already provides many methods
 - `charAt()` , `equals()` , and more on `String`
 - `println()` on `System.out`
- Using a method is termed *calling* on the method
- We can define our own methods

Method syntax

```
return_type name (parameters) {  
    Java code blocks or statements  
}
```

Methods:

- Have a name (required)
- Have inputs called *parameters* (optional)
 - Parameters are variables for argument data
- Return the results of the methods work (optional)
 - Indicating the return type is required
- Code that performs the method work (required)
 - Defined inside of `{ }`

Method names

- Describe the action performed by the method
- Names are case sensitive
 - `computeTip` **!=** `COMPUTETIP`
- Method names follow *lower camel case* conventions
 - Start with a lower case character
 - Name words are joined together
 - Capitalize the first letter of each word

```
double computeTip ( )  
{  
    // method code goes here  
}
```


Method parameters

- Input data is provided to methods via parameters
- Parameters are provided inside of parentheses
 - Provide as many as needed - separate each by a comma
- Use empty parentheses `()` to indicate no parameters
- Parameters are variables for the method code
 - Each parameter is typed like a variable

```
double computeTip (double billAmount, double tipPercentage)
{
    // method code goes here
}
```

Parameter example

- Recall the `charAt(int index)` method

```
"HelloWorld".charAt(5);
```

- `int index` is the **parameter** for the `charAt` method.
- `5` is the **argument** value assigned to `index`

Method return values

- Methods often return the results of their work - the *return value*
 - Return values are also typed (`double` here)

```
double computeTip (double billAmount, double tipPercentage)
{
    // method code goes here
}
```

- Use `void` when there is no return value

```
void soundAlarm () {
    // method code here, but doesn't return a value
}
```

- `main` uses `void` to say it return no value

How to return

- Use `return` with a value to return the method's results
 - Its the last line of a method
- The value must match the return type
- No `return` when `void` is the return type

```
double computeTip (double billAmount, double tipPercentage)
{
    // method code goes here
    return 20;
}
```

A complete method example

```
double computeTip (double billAmount, double tipPercentage) {  
    double tip = billAmount * tipPercentage;  
    return tip;  
}
```

- Return value type - double
- Method name - computeTip
- Parameters (billAmount and tipPercentage) in ()
- Code block in { }
- Use the keyword return to send back the output
 - return is not required when the return type is void

Calling methods

```
public class HelloWorld {  
    public static void main (String[] args) {  
        double smallTip = computeTip(55.0, 0.20); //compute a 20 percent tip on 55 dollars  
        double bigTip = computeTip(135.50, 0.33); //compute a 33 percent tip on 135 dollars  
        System.out.println("Small tip: $" + smallTip);  
        System.out.println("Nice tip: $" + bigTip);  
    }  
    static double computeTip (double billAmount, double tipPercentage) {  
        double tip = billAmount * tipPercentage;  
        return tip;  
    }  
}
```

```
Small tip: $11.0  
Nice tip: $44.715
```

Let's practice!

INTRODUCTION TO JAVA